

数字系统设计

Digital System Design

-----可编程器件与硬件描述语言

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信网络工程系

Zhejiang University

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室（可以钉钉、微信或邮件联系）.

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社,2016.
 - 王金明著,《数字系统设计与Verilog HDL》,电子工业出版社,第8版
 - 《数字系统设计-补充讲义》,浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition,电子工业出版社,2005.
 - M.M.Mano, 数字设计(第四版),电子工业出版社,2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**

- 期中考试 **10%**
- Homework Sets **15% (12~20)%**
- Project **5% (5~7)%**
- Class Room Check+小测验等 **10% (7~10)%**
- 作业每周一上交, **截止期为课后一周内**有效。

- **Project**

- Project-1必做 (1 members)
- Project-2可选(成绩加分1%~5%, 但不超过平时成绩范围)

- Finial Exam期末**闭卷**考试 - **60%**

授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

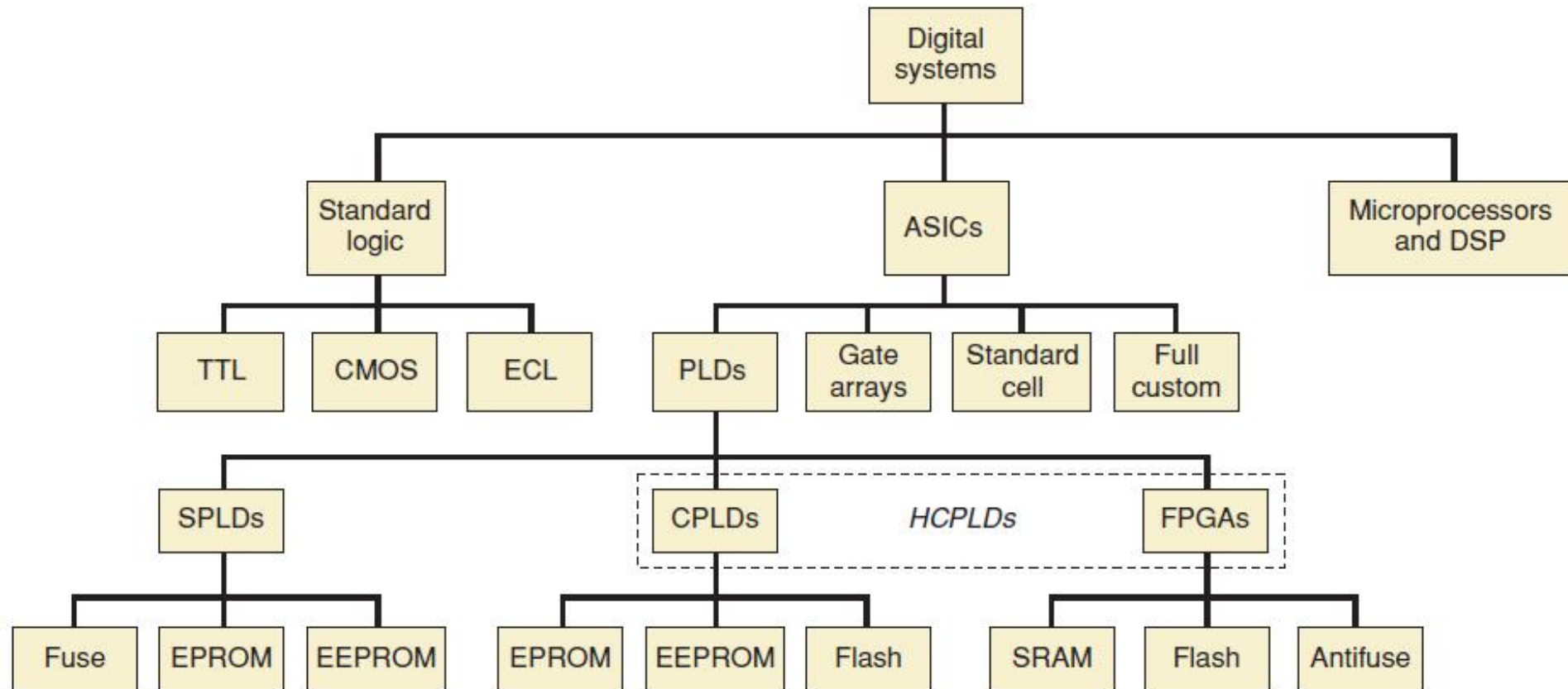
- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

4.6 可编程逻辑器件

Today, programmable logic devices, which contain the circuitry necessary to create logic functions, are being used to implement digital systems.

Why have PLDs taken over so much of the market? With programmable devices, the same functionality can be obtained with one IC rather than using several individual logic chips. This characteristic means less board space, less power required, greater reliability, less inventory, and overall lower cost in manufacturing.

DIGITAL SYSTEMS FAMILY TREE

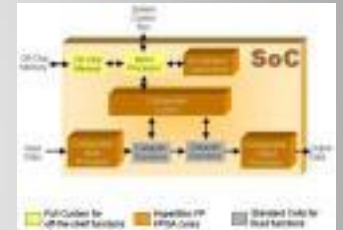
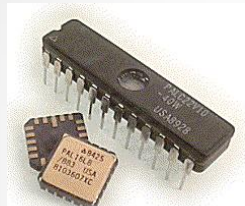


4.6 可编程逻辑器件 (PLD, Programmable Logic Device)

4.6.1 概述

一、PLD的基本特点

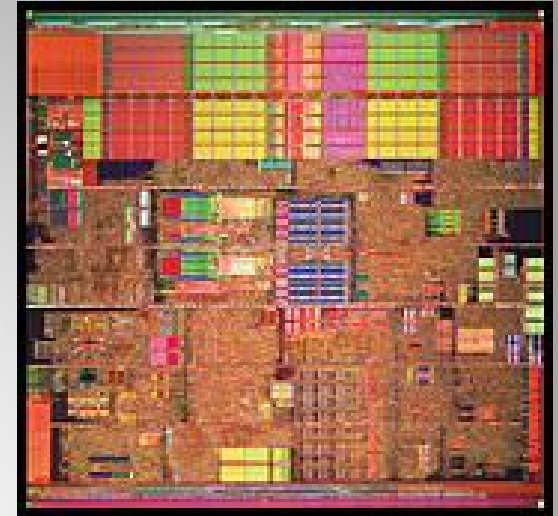
1. 数字集成电路从功能上分为通用型、专用型两大类



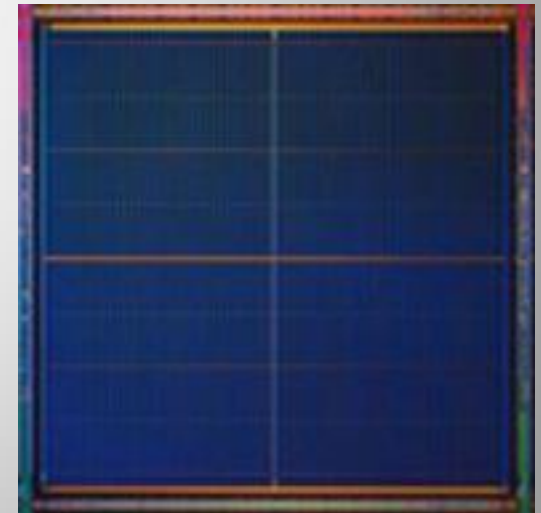
2. PLD的特点：是一种按通用器件来生产，但逻辑功能是由用户通过对器件编程来设定的

Programmable Logic

- **Regular logic**
 - Programmable Logic Arrays
 - Multiplexers/Decoders
 - ROMs
- **Field Programmable Gate Arrays**
 - Xilinx Vertex



“Random Logic”
Full Custom Design



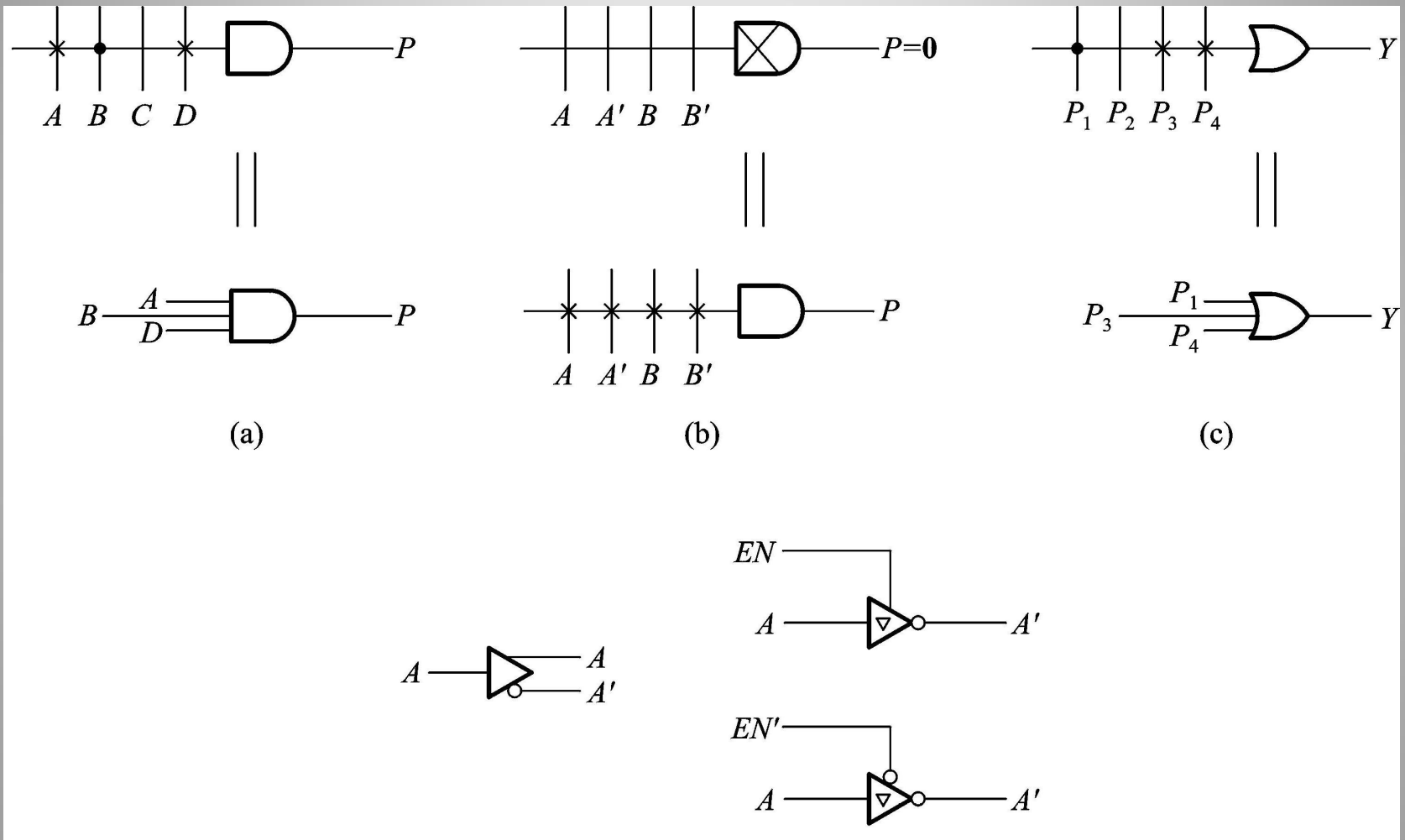
“Regular Logic”
Structured Design

二、PLD的发展和分类

PROM是最早的PLD

1. PAL 可编程逻辑阵列
2. FPLA 现场可编程阵列逻辑
3. GAL 通用阵列逻辑
4. EPLD 可擦除的可编程逻辑器件
5. FPGA 现场可编程门阵列
6. ISP-PLD 在系统可编程的PLD

三、LSI中用的逻辑图符号



4.6.2 现场可编程逻辑阵列 FPLA Field Programmable Logic Array

组合电路和时序电路结构的通用形式1970s

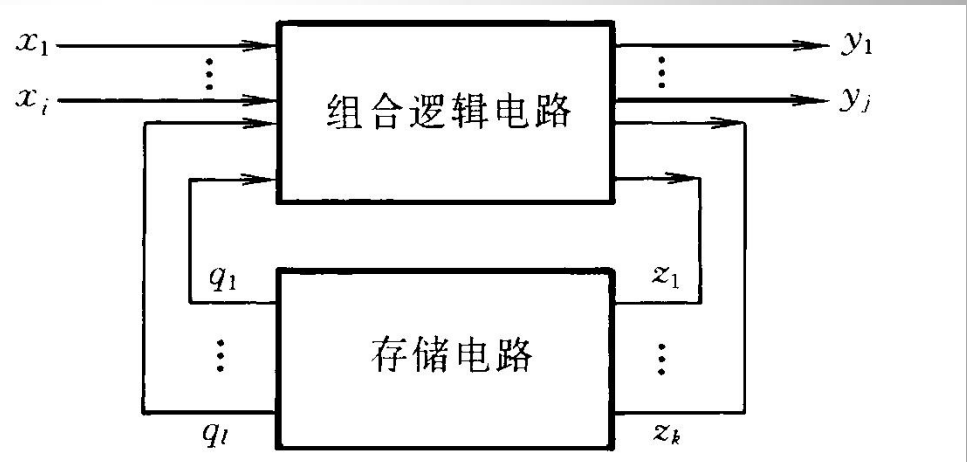
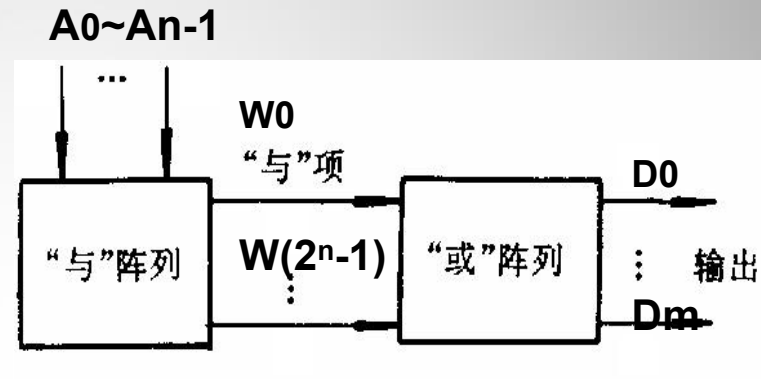
逻辑函数→与或表达式

→与逻辑+或逻辑

→最小项之和

→部分最小项

与EPROM很相似

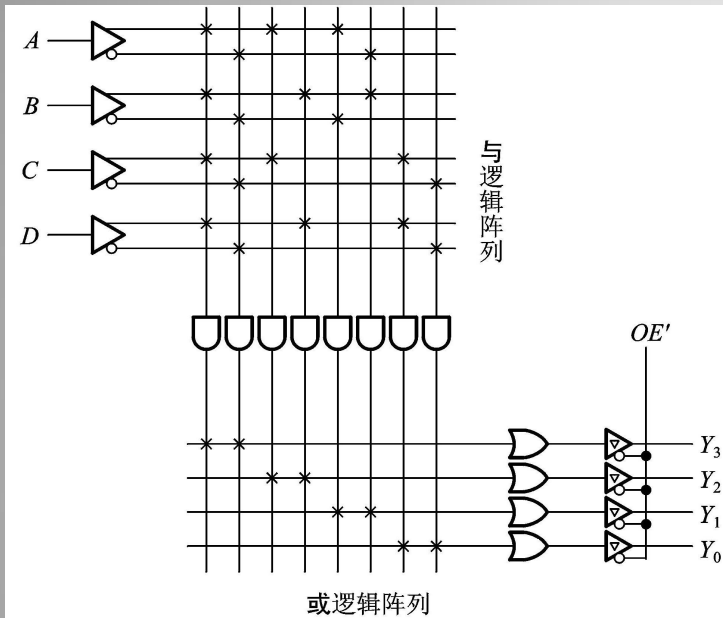


时序型FPLA

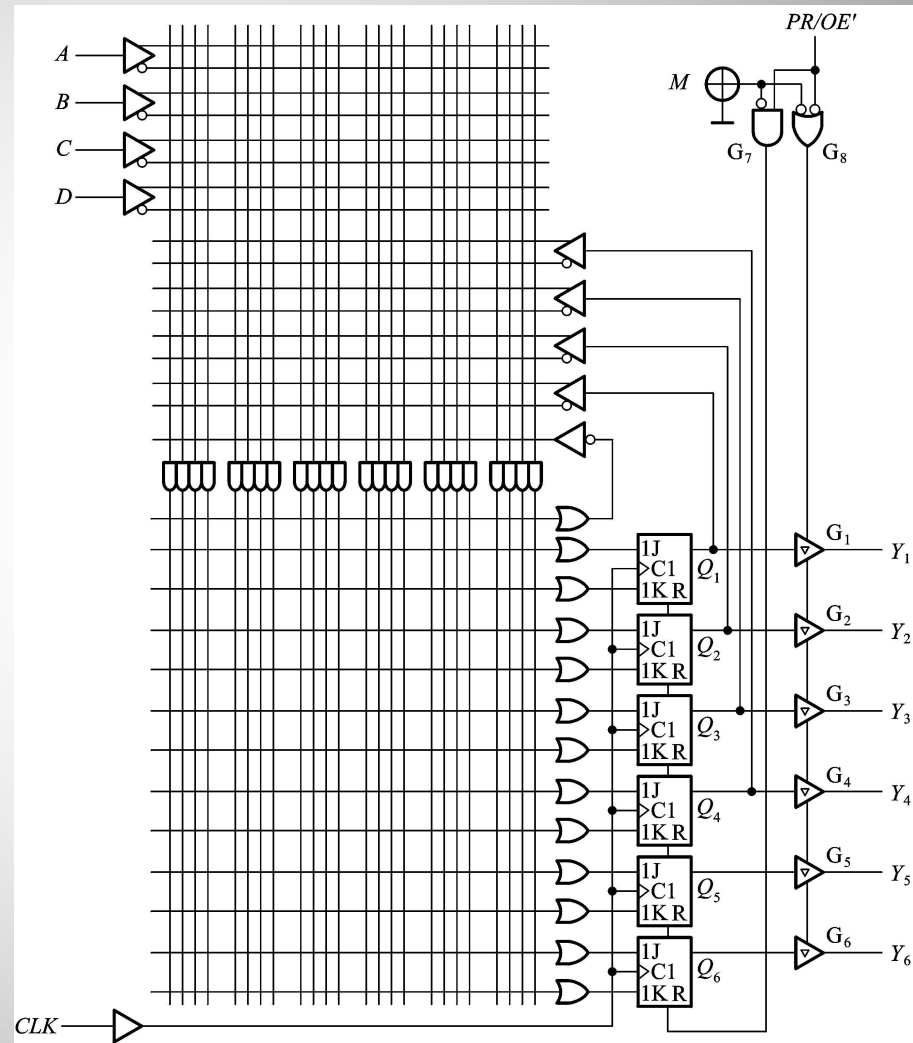
Although the FPLA is more flexible than the PAL architecture, it has not been as widely accepted by engineers. FPLAs are used mostly in state-machine design where a large number of product terms are needed in each SOP expression.

4.6.2 FPLA结构

组合电路和时序电路结构的通用形式



可编程的“与”阵列
+ 可编程的“或”阵列



4.6.3 PAL (Programmable Array Logic)

PAL的基本电路结构，@1970s，曾大规模应用，
采用双极型熔丝工艺，工作速度较高。

一、基本结构形式

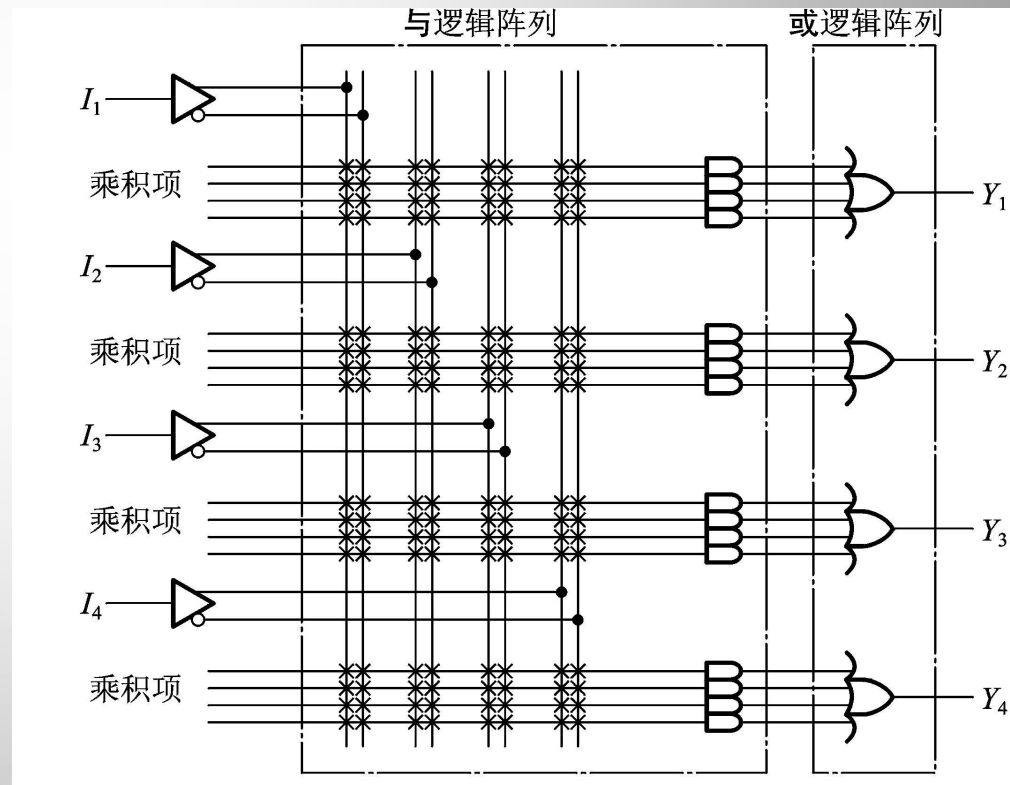
可编程“与”阵列+固定“或”阵列+输出电路

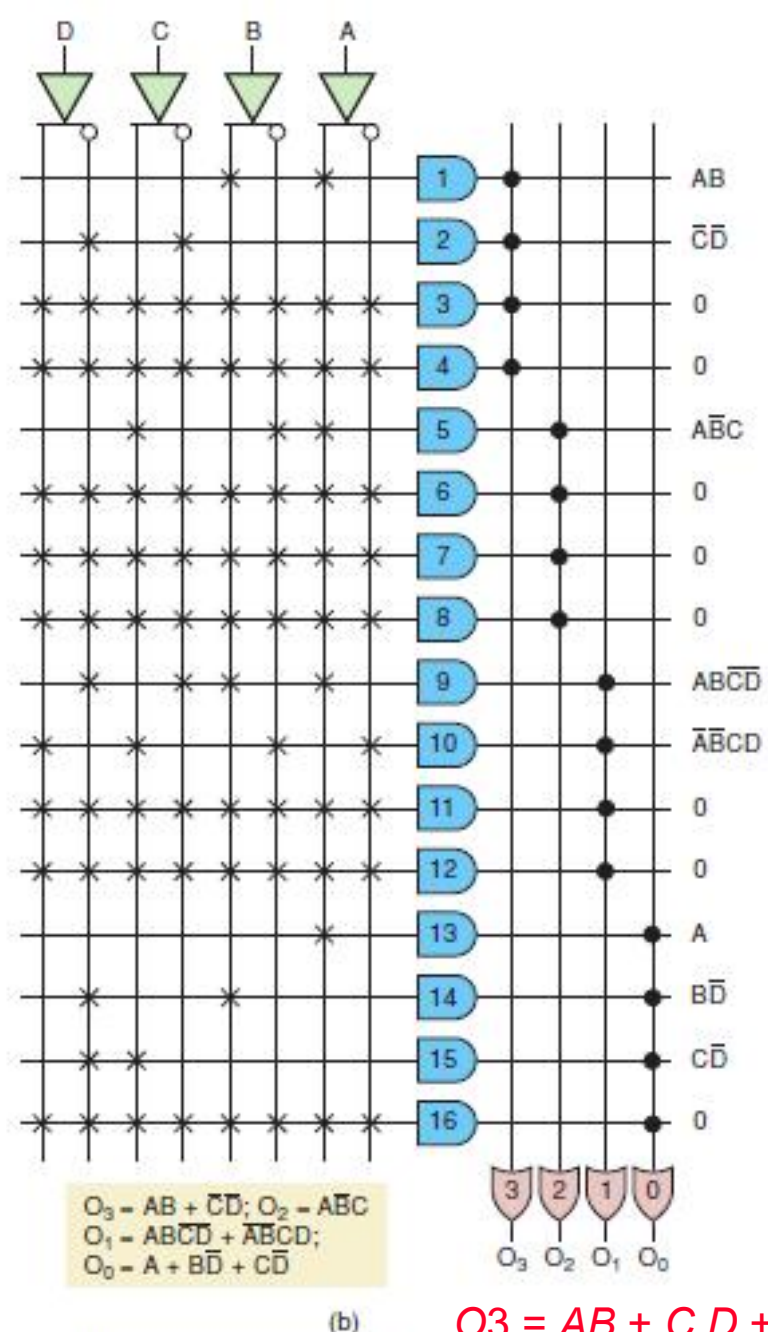
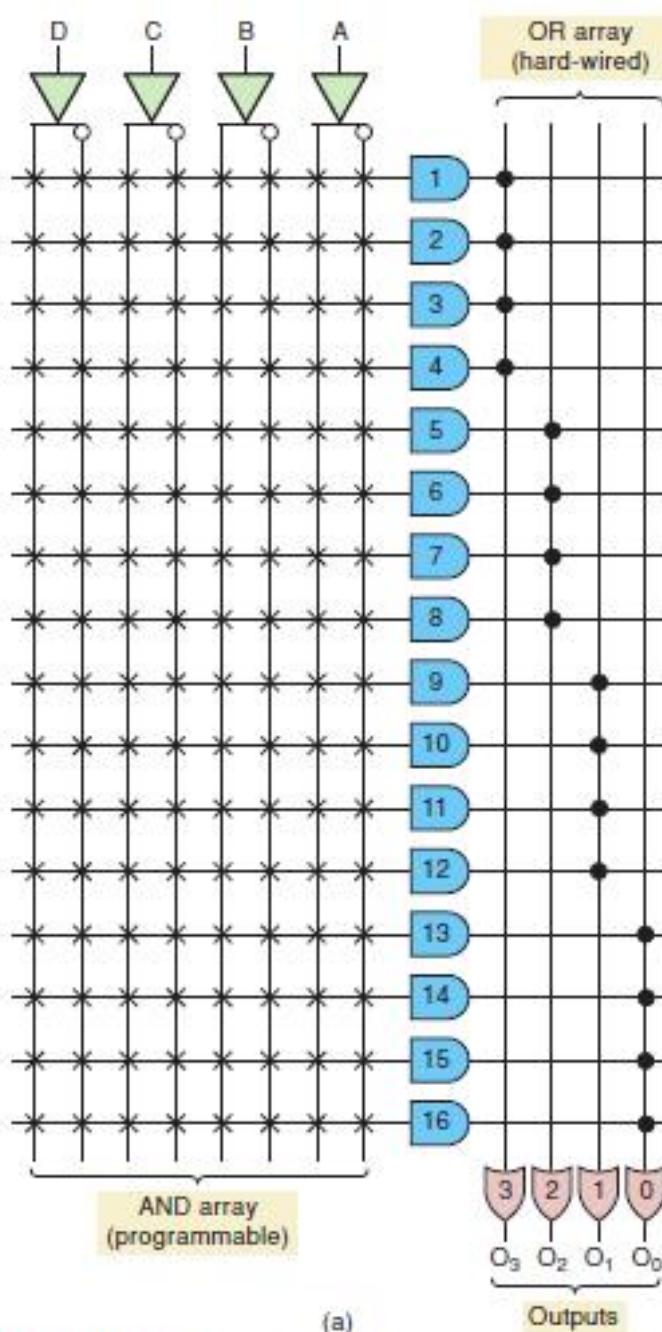
最简单的形式为：

二、编程单元

出厂时，
所有的交叉点均有熔丝

三、输出有限、减少单元数





$$O_3 = AB + \bar{C}D + 0 + 0$$

FIGURE 13-6 (a) Typical PAL architecture; (b) the same PAL programmed for the given functions.

4.6.4 通用逻辑阵列 GAL

电路结构形式@1985

采用**E²CMOS**工艺和灵活的输出结构，有电擦写反复编程的特性。

可编程“与”阵列 + 固定“或”阵列 + 可编程输出电路

OLMC

- ✓ 编程单元
- ✓ 采用E²CMOS 可改写
- ✓ 统一型号

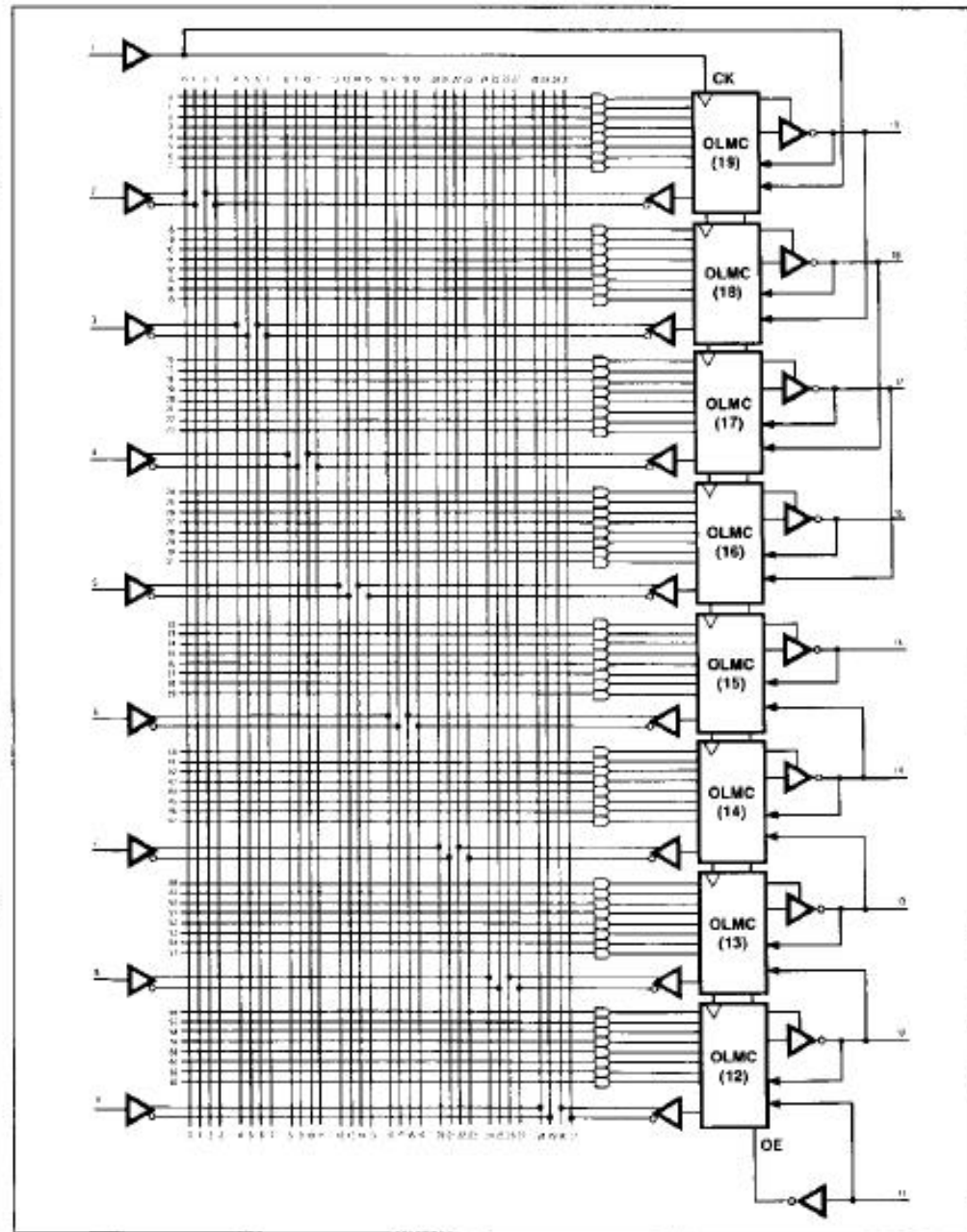
输出逻辑宏单元**OLMC** (Output Logic Macro Cell)

适当地为**OLMC**进行编程，**GAL**就可以在功能上代替前面讨论过的**PAL**各种输出类型以及其衍生类型

GAL 器件结构和特点

GAL16V8的基本结构

GAL16V8 LOGIC DIAGRAM



4.6.5 可擦除的可编程逻辑阵列EPLD

一、结构特点

相当于

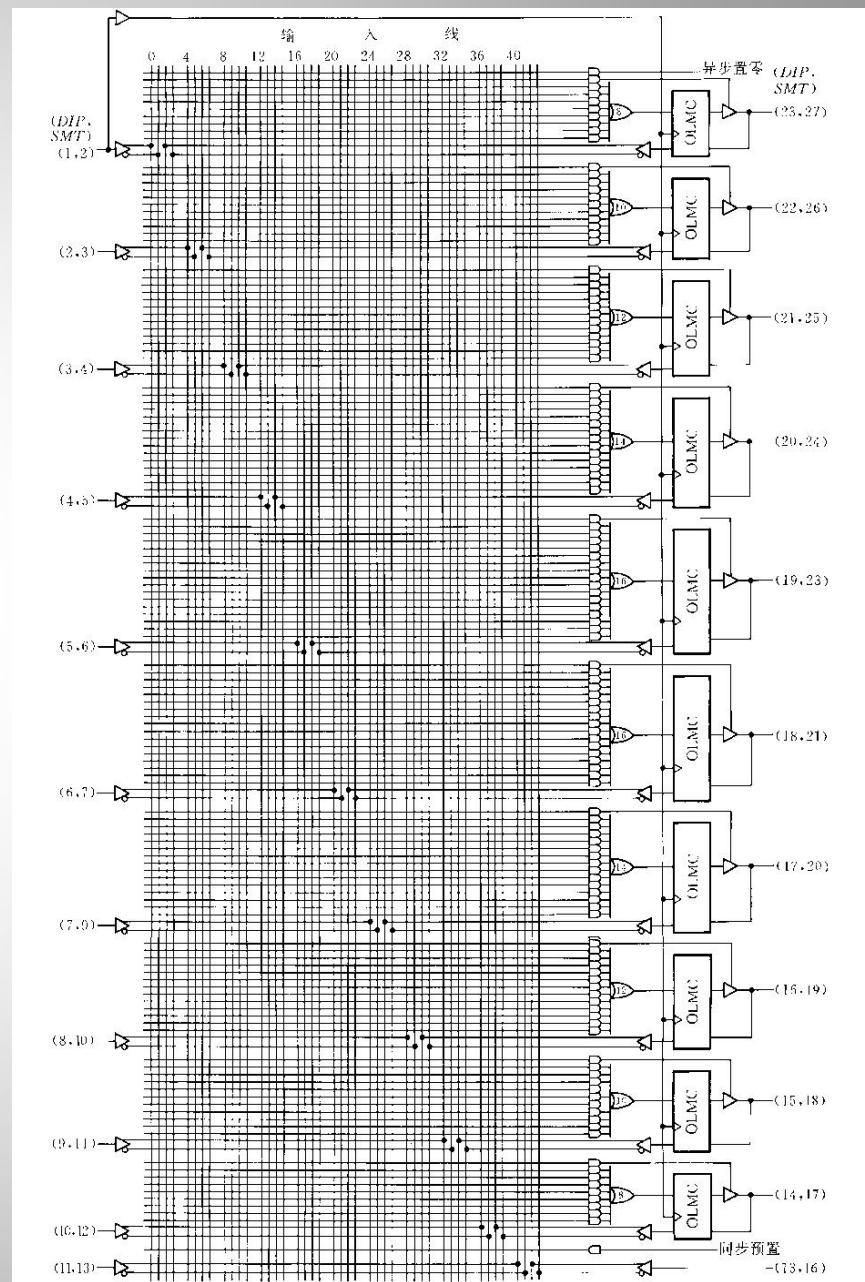
“与-或”阵列 (PAL) + OLMC

二、采用EPROM工艺

集成度提高

高密度

复杂的可编程逻辑阵列CPLD

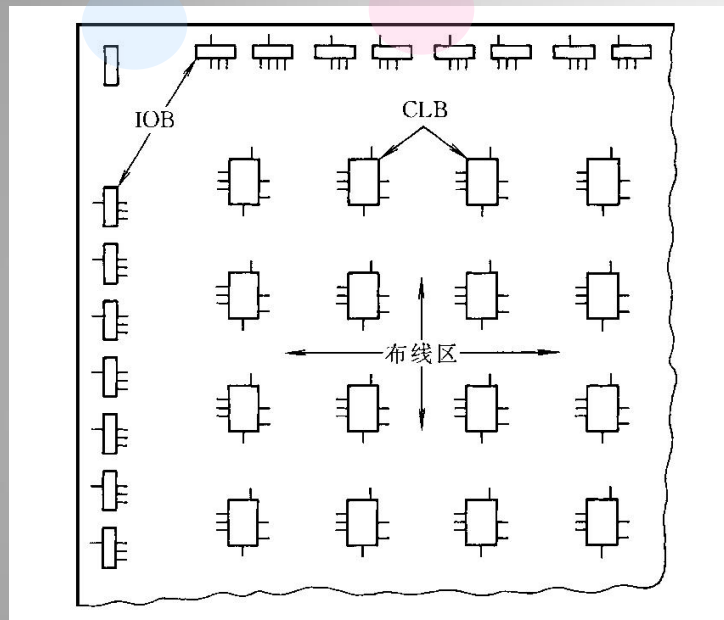


4. 6. 7 现场可编程门阵列FPGA

- Basic idea: two-dimensional array of logic blocks and flip-flops with a means for the user to configure:

- the interconnection between the logic blocks,
- the function of each block.

一、基本结构

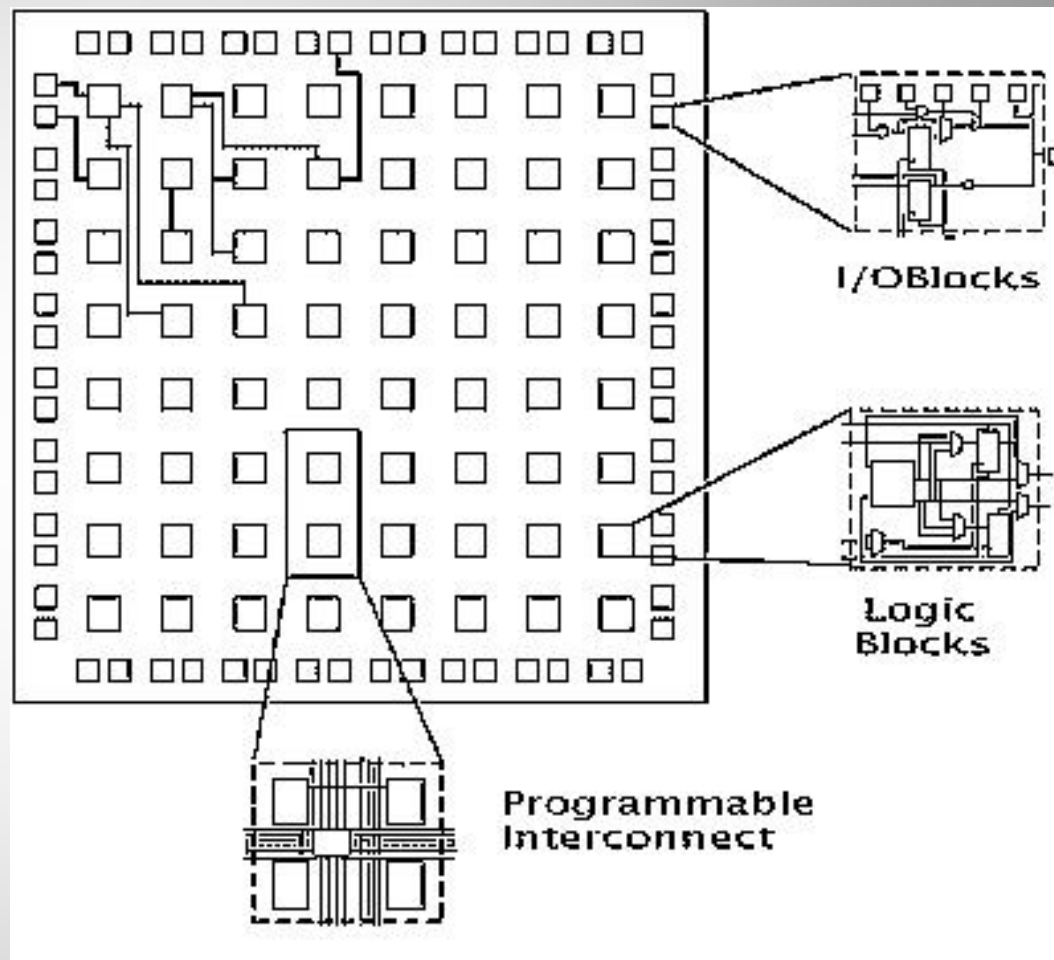


1. IOB输入输出

2. CLB逻辑

3. 互连资源IR

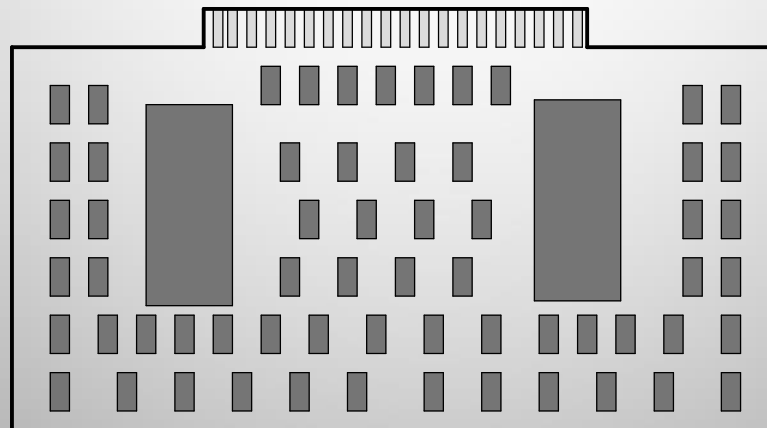
4. SRAM编程数据



Simplified version of FPGA internal architecture

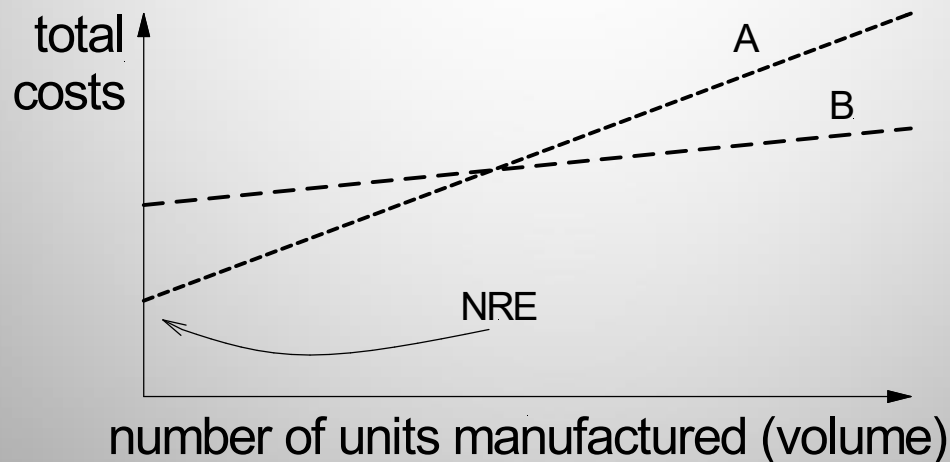
Why FPGAs?

- By the early 1980's most of the logic circuits in typical systems where absorbed by a handful of standard large scale integrated circuits (LSI).
 - Microprocessors, bus/IO controllers, system timers, ...
- Every system still had the need for random “glue logic” to help connect the large ICs:
 - generating global control signals (for resets etc.)
 - data formatting (serial to parallel, multiplexing, etc.)
- Systems had a few LSI components and lots of small low density SSI (small scale IC) and MSI (medium scale IC) components.



Why FPGAs?

- **Custom ICs sometimes designed to replace the large amount of glue logic:**
 - reduced system complexity and manufacturing cost, improved performance.
 - However, custom ICs are very expensive to develop, and delay introduction of product to market (time to market) because of increased design time.
- **Note: need to worry about two kinds of costs:**
 1. cost of development, sometimes called non-recurring engineering (NRE)
 2. cost of manufacture
 - A tradeoff usually exists between NRE cost and manufacturing costs

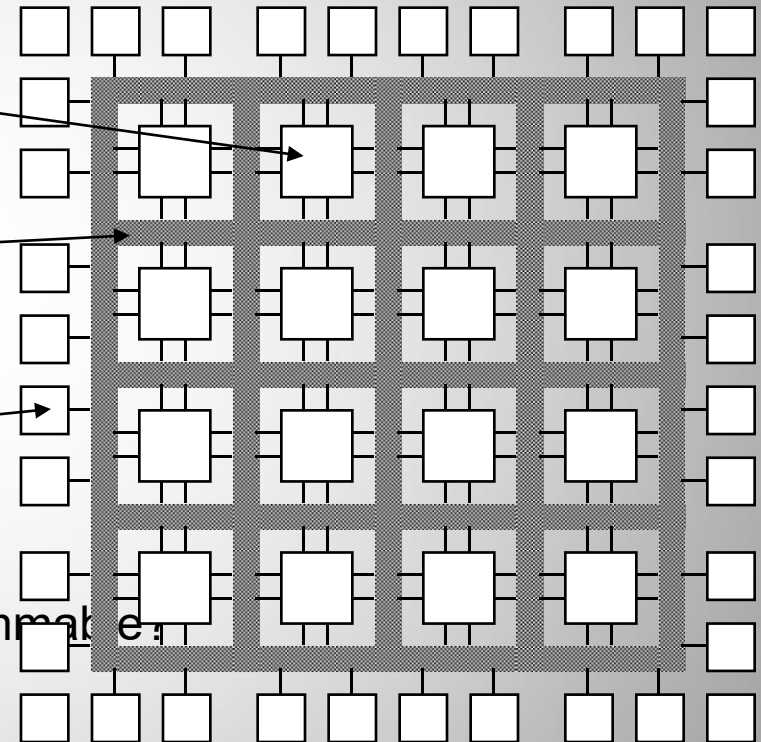


Why FPGAs?

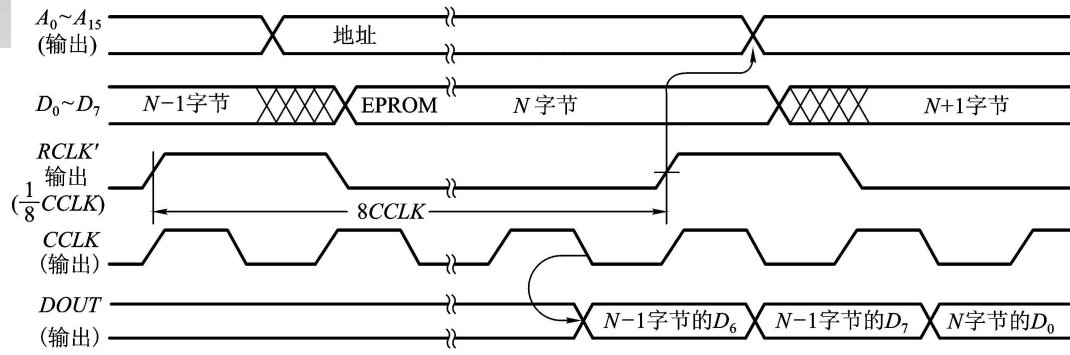
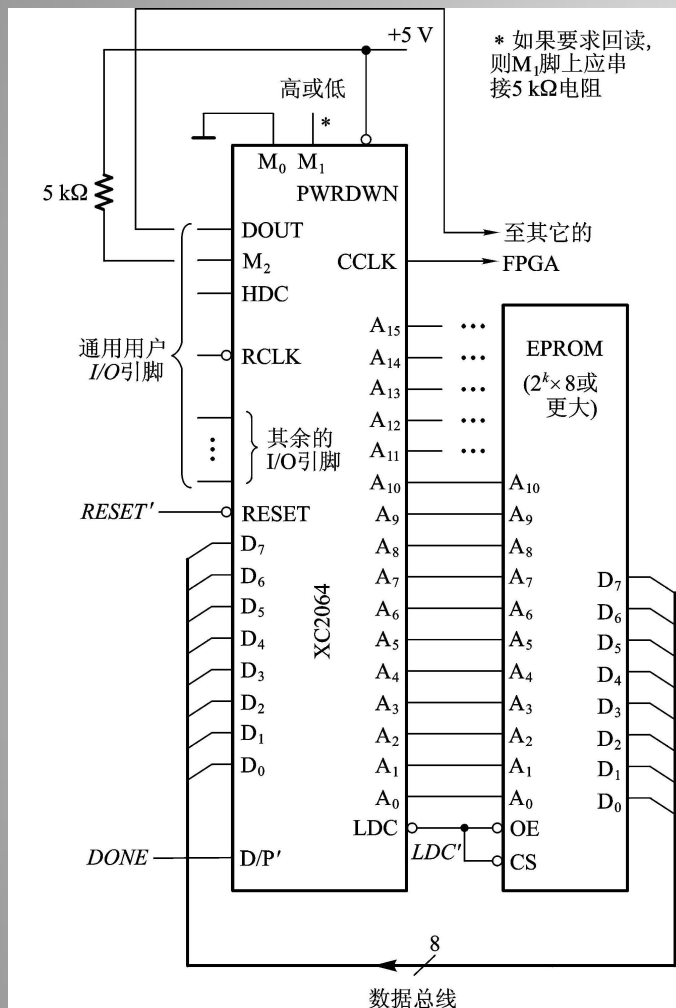
- **Custom IC approach viable for products that are ...**
 - very high volume (where NRE could be amortized),
 - not time-to-market sensitive.
- **FPGAs introduced as an alternative to custom ICs for implementing glue logic:**
 - improved density relative to discrete SSI/MSI components (within around 10x of custom ICs)
 - with the aid of computer aided design (CAD) tools circuits could be implemented in a short amount of time (no physical layout process, no mask making, no IC manufacturing), relative to ASICs.
 - » lowers NREs
 - » shortens TTM
- **Because of Moore's law the density (gates/area) of FPGAs continued to grow through the 80's and 90's to the point where major data processing functions can be implemented on a single FPGA.**

Field-Programmable Gate Arrays

- **PLAs:** 100s of gate equivalents
- **FPGAs:** 1000-10000s gates → upto 10,000,000 gates
- **Logic blocks**
 - Implement combinational and sequential logic
- **Interconnect**
 - Wires to connect inputs and outputs to logic blocks
- **I/O blocks**
 - Special logic blocks at periphery of device for external connections
- **Key questions:**
 - How to make logic blocks programmable?
 - How to connect the wires?
 - *After the chip has been fabbed*



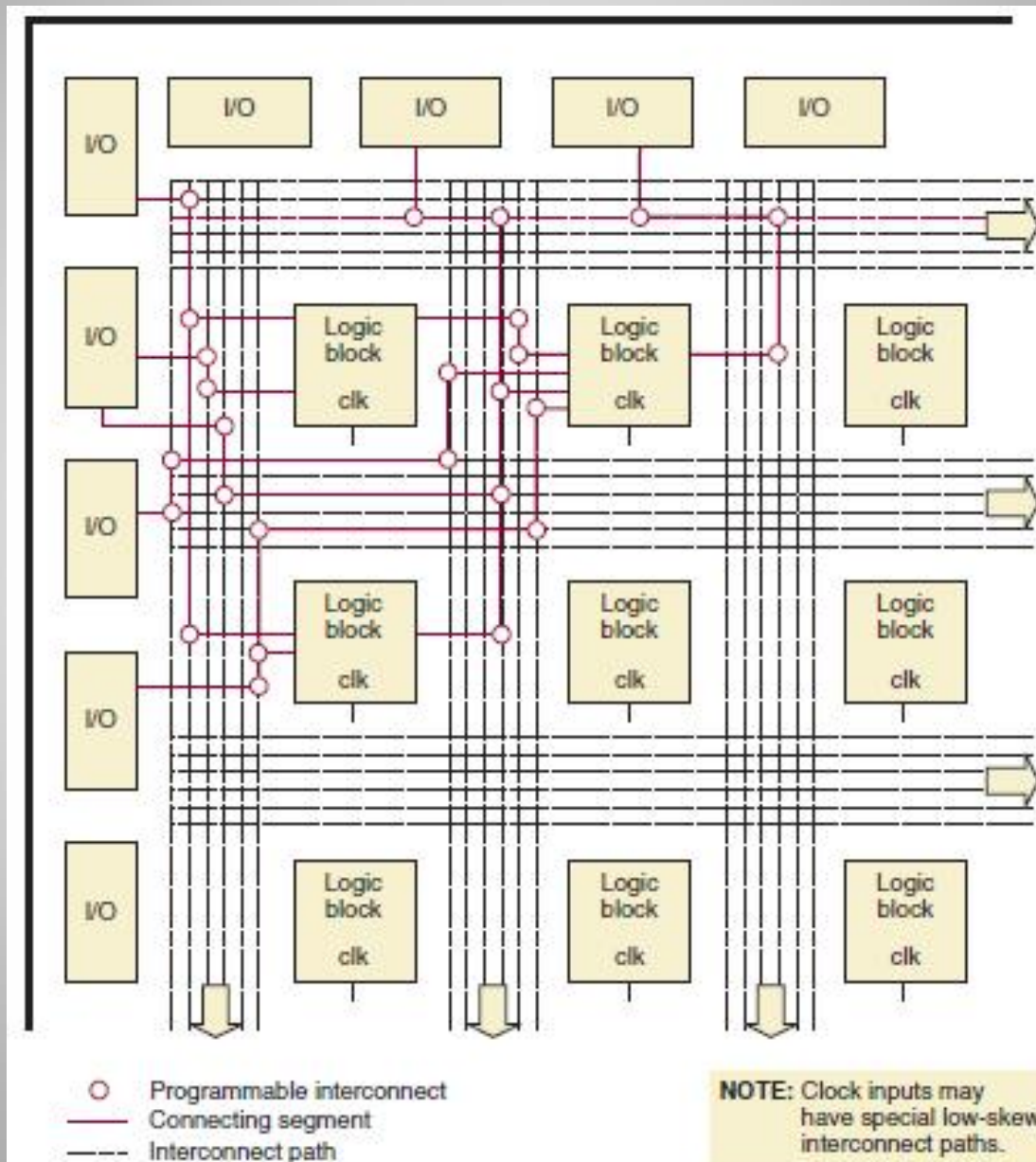
4.6.8 编程数据的装载



1. 数据可先放在EPROM或PC机中
2. 通电后, 自行启动FPGA内部的一个时序控制逻辑电路, 将在EPROM中存放的数据读入FPGA的SRAM中
3. “装载”结束后, 进入编程设定的工作状态

!! 每次停电后, SRAM中数据消失
下次工作仍需重新装载

FPGA architecture



4.6.9 PLD的编程

以上各种PLD均需离线进行编程操作，使用开发系统

一、开发系统

1. 硬件：计算机+编程器
2. 软件：开发环境（软件平台）

VHDL, Verilog

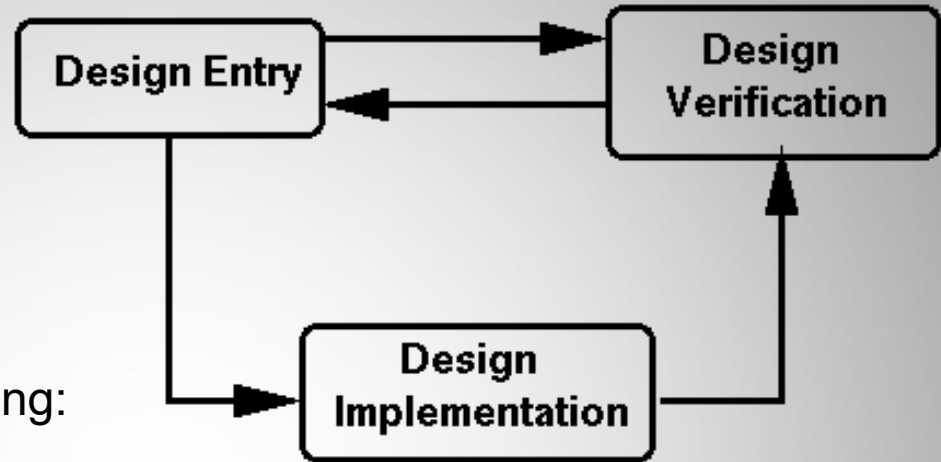
真值表，方程式，电路逻辑图（Schematic）

状态转换图（FSM）

二、步骤

- ① 抽象（系统设计采用Top-Down的设计方法）
- ② 选定PLD
- ③ 选定开发系统
- ④ 编写源程序（或输入文件）
- ⑤ 调试，运行仿真，产生下载文件
- ⑥ 下载
- ⑦ 测试

FPGA Generic Design Flow



- **Design Entry:**
 - Create your design files using:
 - » schematic editor or
 - » hardware description language (Verilog, VHDL)
- **Design “implementation” on FPGA:**
 - *Partition, place, and route* to create bit-stream file
- **Design verification:**
 - Use Simulator to check function,
 - other software determines max clock frequency.
 - Load onto FPGA device (cable connects PC to development board)
 - » check operation at full speed in real environment.

JTAG interface between PC parallel port and EPM7128SLC84

- programmer

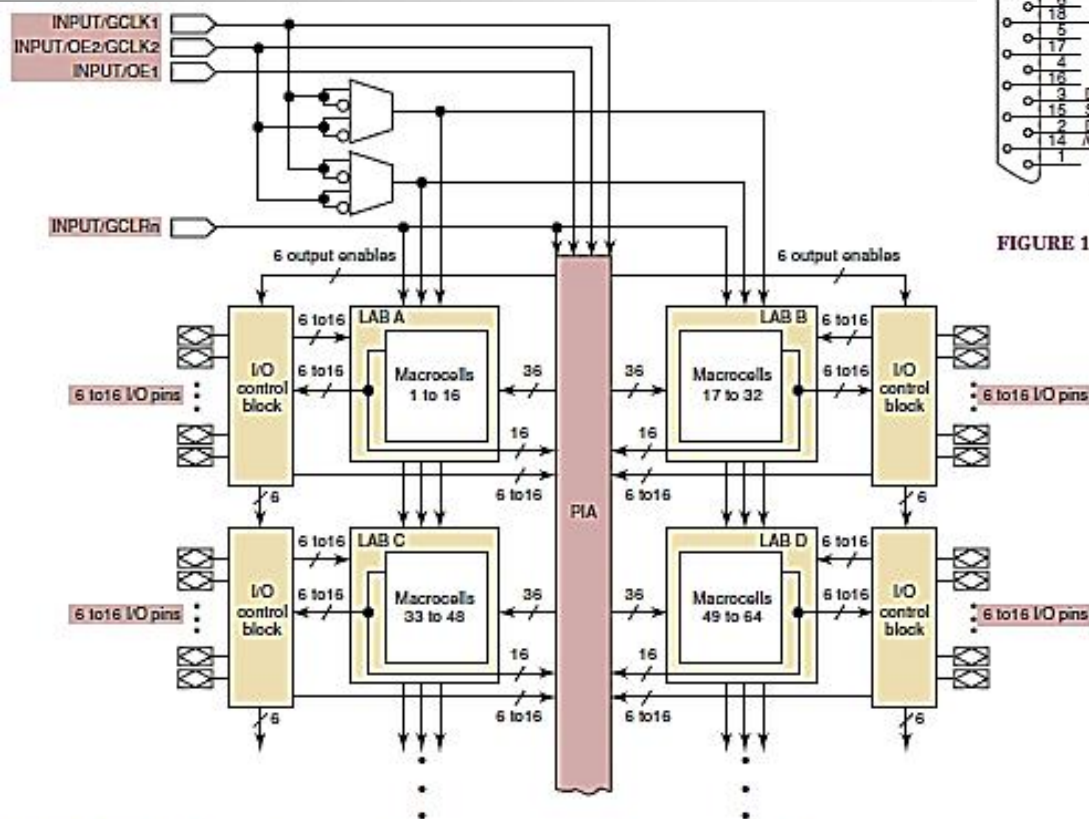


FIGURE 13-10 MAX7000S family block diagram. (Courtesy of Altera Corporation.)

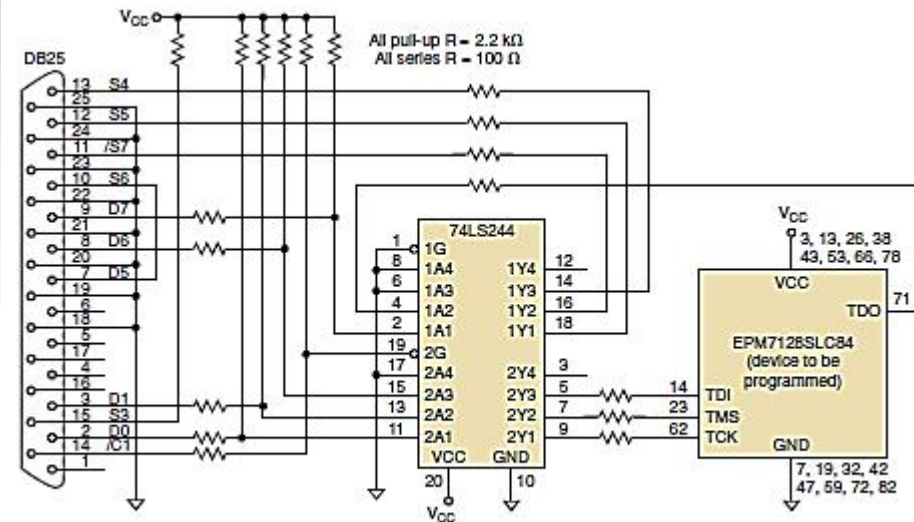


FIGURE 13-11 JTAG interface between PC parallel port and EPM7128SLC84.

4.7 硬件描述语言Verilog HDL简介

---@组合电路设计

•概述

一、什么是Verilog HDL?

Verilog HDL是一种应用广泛的硬件描述性语言，主要用于从算法级、门级到开关级的多种抽象层次的数字系统建模。

二、为什么要用Verilog HDL?

- 电子设计规模越来越大（普通设计已达几百万门的数量级），复杂度越来越高。有必要用高级语言来表达其功能，隐藏其具体的细节实现。
- 提高逻辑设计的效率，降低设计成本，更重要的是缩短设计同期。
- 可读性强，易修改。

Verilog HDL的基本结构:

简单的Verilog HDL的例子

例1 一个8位加法器的Verilog HDL源代码

准备实现的逻辑功能:

```
module adder8 (cout, sum, ina, inb, cin);
```

```
    output[7:0] sum;
```

```
    output cout;
```

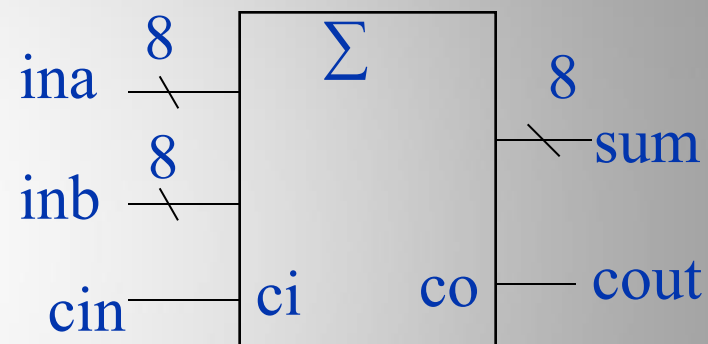
```
    input[7:0] ina, inb;
```

```
    input cin;
```

```
    assign {cout, sum} = ina + inb + cin;
```

```
endmodule
```

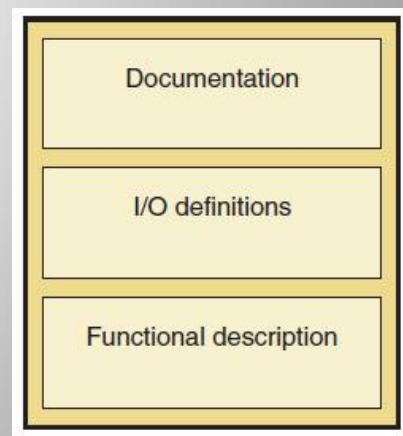
输入、输出端口描述



程序为模块结构, 包含在
module与**endmodule**间

逻辑功能描述

问题: 若要设计16位加法器, 怎样修改?



例2 8位二进制加法计数器

```
module counter8 (out, cout, data, load, cin, clk);
```

```
    output [7:0] out;
```

```
    output cout;
```

```
    input load, cin, clk;
```

```
    input [7:0] data;
```

```
    reg [7:0] out;
```

输入、输出端口描述

准备实现的
逻辑功能：

```
    always @(posedge clk) -- 时钟上沿触发
```

```
    begin
```

```
        if (load) out <= data; // 置数
```

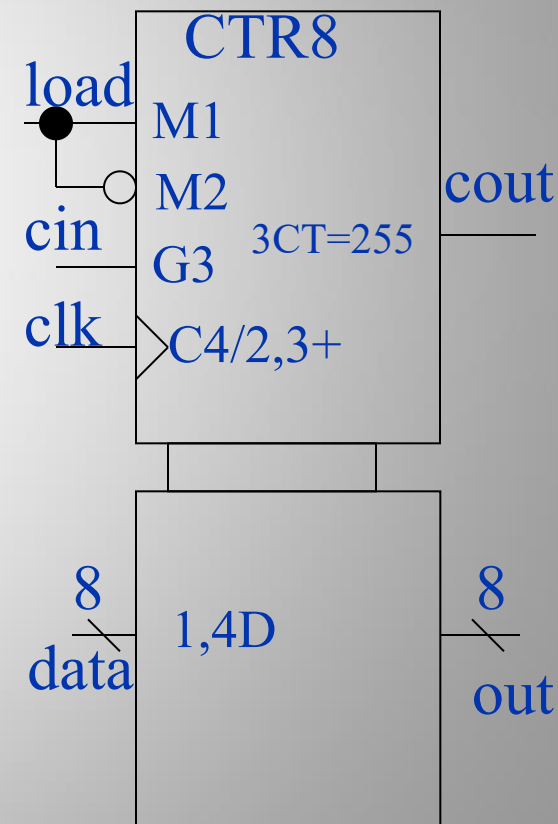
```
    else
```

```
        out <= out + cin; // 计数或保持
```

```
    end
```

```
    assign cout = &out & cin; // 进位
```

```
endmodule
```



Verilog HDL模块的结构

1.模块的概念

- 模块是Verilog HDL语言的基本单元，数字系统是用模块集合的形式来描述。
- 模块是描述某个设计的功能、结构和与其它模块通信的外部端口。
- Verilog HDL中各个模块是并行运行。
- 模块可以调用其它模块的实例。

2.模块的结构

module <模块名>(<端口列表>)

端口说明 (**input**, **output**, **inout**)

参数定义 (可选)

数据类型定义 //**wire,reg,task,function**

连续赋值语句 (**assign**) //组合逻辑

过程块 (**always**和**initial**)

-行为描述语句

低层模块实例 //调用其它模块

任务和函数

延时说明块

endmodule

语句的顺序执行与并行执行

- 一、**assign**语句之间：并行执行（同时执行）
- 二、过程块之间（**always**, **initial**）：并行执行
- 三、**assign**语句与过程块之间：并行执行
- 四、过程块（**always**, **initial**）内部

非阻塞语句类似并行语句

- 
- 串行块**（**begin-end**）：顺序执行
 - 并行块**（**fork-join**）：并行执行

6) 位运算符(~,|,^,&,&~)

- 1) ~ //取反
- 2) & //按位与
- 3) | //按位或
- 4) ^ //按位异或
- 5) ^~ //按位同或(异或非)

说明:

- 位运算符中除了~是单目运算符以外,均为二目运算符,即要求运算符两侧各有一个操作数.
- 位运算符中的二目运算符要求对两个操作数的相应位进行运算操作。

位拼接运算符(Concatation)

在Verilog HDL语言有一个特殊的运算符：位拼接运算符`{}`。用这个运算符可以把两个或多个信号的某些位拼接起来进行运算操作。其使用方法如下：

{信号1的某几位, 信号2的某几位, ..., 信号n的某几位}

即把某些信号的某些位详细地列出来，中间用逗号分开，最后用大括号括起来表示一个整体信号。见下例：

{a,b[3:0],w,3'b101}

也可以写成为

{a,b[3],b[2],b[1],b[0],w,1'b1,1'b0,1'b1}

在位拼接表达式中不允许存在没有指明位数的信号。

位拼接还可以用重复法来简化表达式。见下例：

{4{w}} //这等同于{w,w,w,w}

位拼接还可以用嵌套的方式来表达。见下例：

{b,{3{a,b}}} //这等同于{b,a,b,a,b,a,b}

缩减运算符(reduction operator)

缩减运算符是单目运算符,也有与或非运算。

例如:

```
reg [3:0] B;
```

```
reg C;
```

```
C = &B;
```

相当于:

```
C =( (B[0]&B[1]) & B[2] ) & B[3];
```

(1).非阻塞(Non_Blocking)赋值方式(如 **b <= a;**)

- 1) 块结束后才完成赋值操作。
- 2) b的值并不是立刻就改变的。
- 3) 这是一种比较常用的赋值方法。(特别在编写可综合模块时)

(2).阻塞(Blocking)赋值方式(如 **b = a;**)

- 1) 赋值语句执行完后,块才结束。
- 2) b的值在赋值语句执行完后立刻就改变的。
- 3) 可能会产生意想不到的结果。

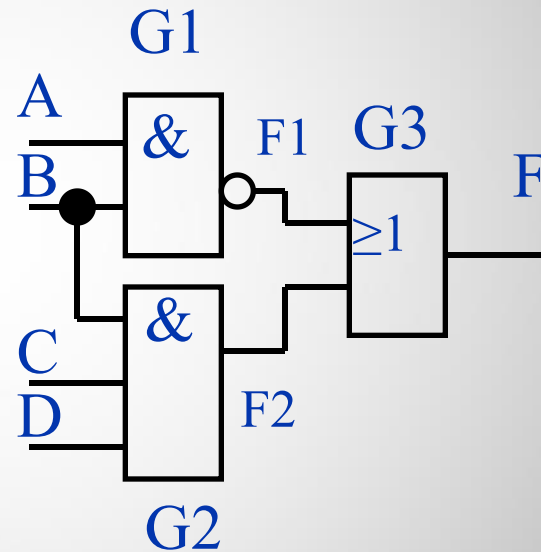
非阻塞赋值方式和阻塞赋值方式的区分常给设计人员带来问题。

常用组合电路的设计

1.基本门电路的描述

方法1: 调用门原语

```
module gate1(F, A, B, C, D);  
  input  A, B, C, D;  
  output F;  
  wire  F1, F2;  
  nand  G1 ( F1,  A,  B  );  
  and   G2 ( F2,  B, C, D);  
  or    G3 ( F,   F1, F2);  
endmodule
```



•调用格式:

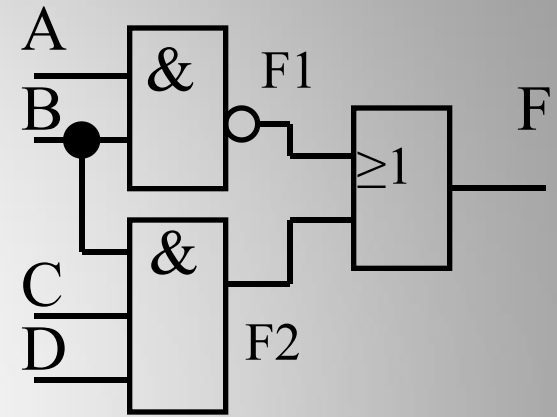
多输入门: 原语名 <例化名称> (输出, 输入1, 输入2.....)

非门: 原语名 <例化名称> (输出, 输入)

方法2: 用assign连续赋值语句描述

```
module gate2(F,A,B,C,D);  
  input A,B,C,D;  
  output F;  
  assign F=~(A & B)|(B & C & D);  
endmodule
```

逻辑表达式



方法3: 用过程赋值语句描述

```
module gate3(F,A,B,C,D);  
  input A,B,C,D;  
  output F;  
  reg F;  
  always @(A or B or C or D)  
  begin  
    F=~(A & B) | (B & C & D);  
  end  
endmodule
```

敏感信号表:
与同步时序电路差别?

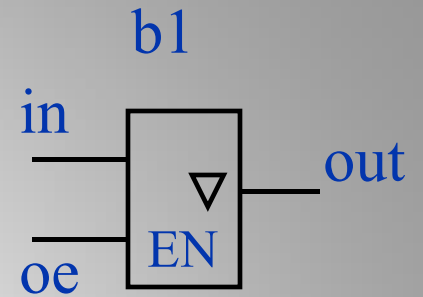
例1 三态门

例 用**bufif1**关键字描述的三态门

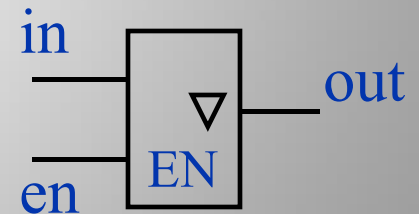
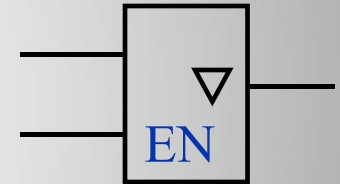
```
module tristate(in, oe, out);  
    input in, oe;  
    output out;  
    tri out;  
    //注意三态门端口的排列顺序  
    bufif1 b1(out, in, oe); //(输入, 输出, 使能)  
endmodule
```

例 用**assign**描述的三态门

```
module tri_1(out, in, en);  
    output out;  
    input in, en;  
    assign out = en ? in : 1'bz;  
    //若en = 1, out = in; 若en = 0, out为高阻态  
endmodule
```



Bufif1原语:

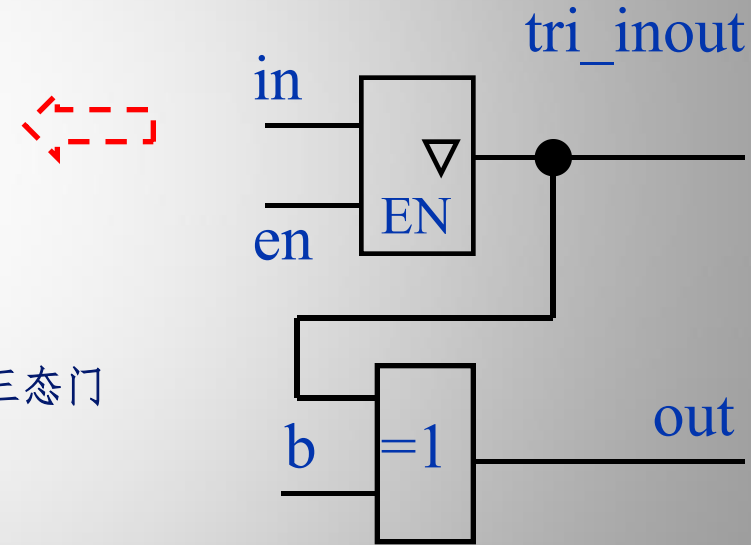


例2 用三态双向驱动器

双向端口

```
module bidir(tri_inout, out, in, en, b);  
    inout tri_inout;  
    output out;  
    input in, en, b;  
    assign tri_inout = en ? in : 1'bz; // 三态门  
    assign out = tri_inout ^ b;
```

endmodule



例3 译码器、编码器

3线_8线译码器（输出低电平有效）

```
module edcoder_38(out, in);
```

```
    output[7:0] out;
```

```
    input[2:0] in;
```

```
    reg[7:0] out;
```

```
    always @(in)
```

```
    begin
```

```
        case(in)
```

```
            3'd0: out = 8'b1111_1110;
```

```
            3'd1: out = 8'b1111_1101;
```

```
            3'd2: out = 8'b1111_1011;
```

```
            3'd3: out = 8'b1111_0111;
```

```
            3'd4: out = 8'b1110_1111;
```

```
            3'd5: out = 8'b1101_1111;
```

```
            3'd6: out = 8'b1011_1111;
```

```
            3'd7: out = 8'b0111_1111;
```

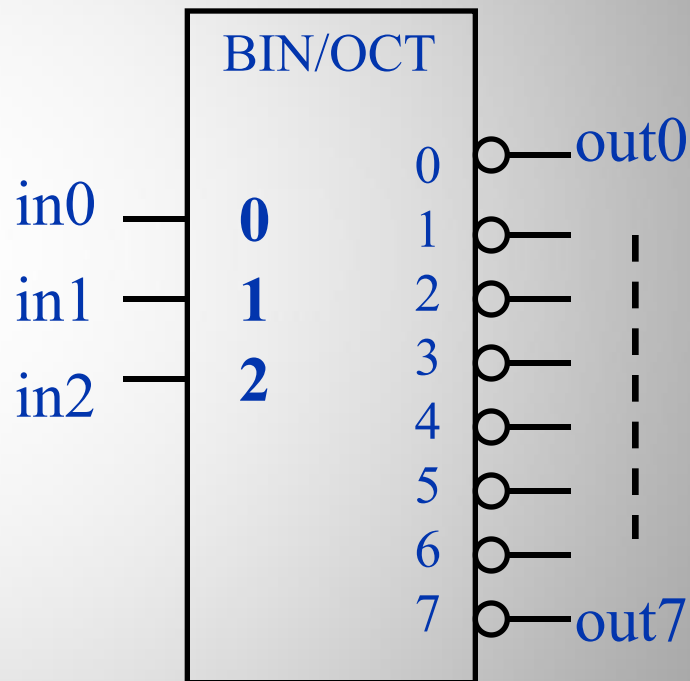
```
        endcase
```

```
    end
```

```
endmodule
```

分隔符，便于阅读

真值表



例4 8线_3线高优先编码器

方法1: 用if_else语句描述编码器

```
module encoder8_3(none_on,outcode,a, b, c, d, e, f, g, h);
```

```
output[2:0] outcode;
```

```
output none_on;
```

```
input a, b, c, d, e, f, g, h;
```

```
reg[3:0] outtemp;
```

```
assign {none_on, outcode} = outtemp;
```

```
always @(a or b or c or d or e or f or g or h)
```

```
begin
```

```
    if(h) outtemp = 4'b0_111;
```

```
    else if(g) outtemp = 4'b0_110;
```

```
    else if(f) outtemp = 4'b0_101;
```

```
    else if(e) outtemp = 4'b0_100;
```

```
    else if(d) outtemp = 4'b0_011;
```

```
    else if(c) outtemp = 4'b0_010;
```

```
    else if(b) outtemp = 4'b0_001;
```

```
    else if(a) outtemp = 4'b0_000;
```

```
    else outtemp = 4'b1_000;
```

```
end
```

```
endmodule
```

引进变量，便于描述

BIN/OCT		
a	0	none_on
b	1	(输入无效)
c	2	
d	3	
e	4	
f	5	0 — outcode0
g	6	1 — outcode1
h	7	2 — outcode2

方法2：用case语句描述编码器

```
module encoder_83(none_on,outcode,a, b, c, d, e, f, g, h);
```

```
    output[2:0] outcode;
```

```
    output    none_on;
```

```
    input     a, b, c, d, e, f, g, h;
```

```
    reg[3:0]  outtemp;
```

```
    assign {none_on, outcode} = outtemp;
```

```
    always @(a or b or c or d or e or f or g or h)
```

```
    begin
```

```
        casex ({a, b, c, d, e, f, g, h})
```

```
            8'B????_???1 : outtemp=4'b0_111;
```

```
            8'B????_??10 : outtemp=4'b0_110;
```

```
            8'B????_?100 : outtemp=4'b0_101;
```

```
            8'B????_1000 : outtemp=4'b0_100;
```

```
            8'B???1_0000 : outtemp=4'b0_011;
```

```
            8'B??10_0000 : outtemp=4'b0_010;
```

```
            8'B?100_0000 : outtemp=4'b0_001;
```

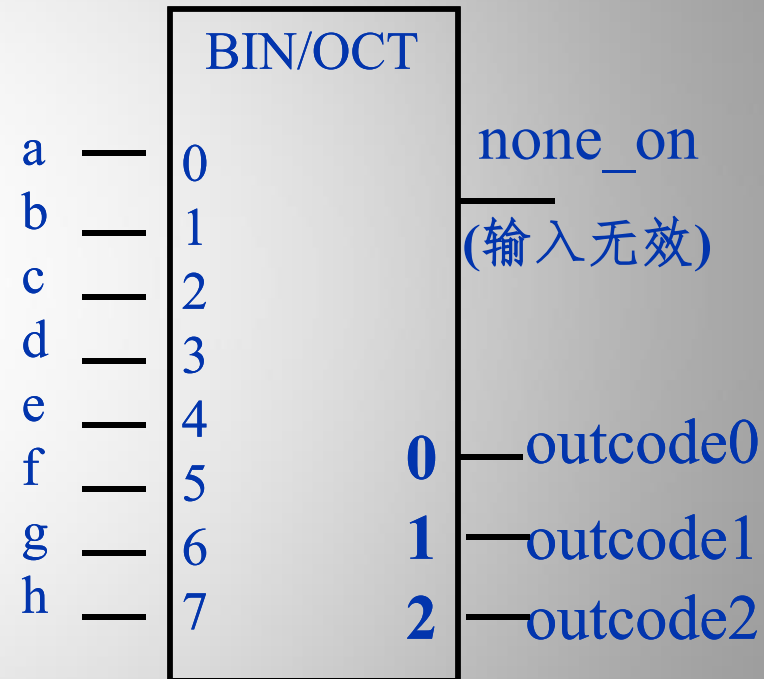
```
            8'B1000_0000 : outtemp=4'b0_000;
```

```
            8'B0000_0000 : outtemp=4'b1_000;
```

```
        endcase
```

```
    end
```

```
endmodule
```

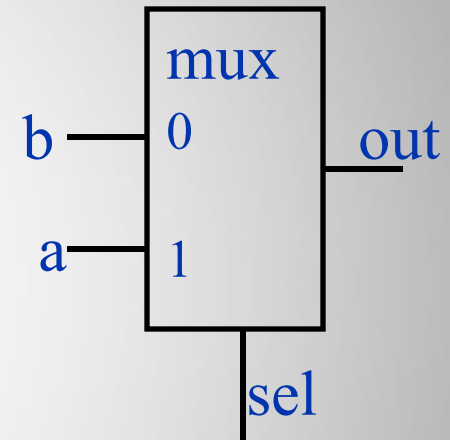


例5 数据选择器MUX

用**assign**语句设计2选1 MUX

```
module mux2_1(out, a, b, sel);  
    output out;  
    input  a, b, sel;  
    assign (out = sel ? a : b);  
endmodule
```

sel=1: out=a
sel=0: out=b



用if-else语句描述4选1 MUX

```
module mux4_1(out , in0 , in1 , in2 , in3 , sel);
```

```
    output out ;
```

```
    input[7:0] in0 , in1 , in2 , in3 ;
```

```
    input[1:0] sel;
```

```
    reg[7:0] out ;
```

```
    always @(in0 or in1 or in2 or in3 or sel)
```

```
    begin
```

```
        if(sel== 2'b00)      out = in0;
```

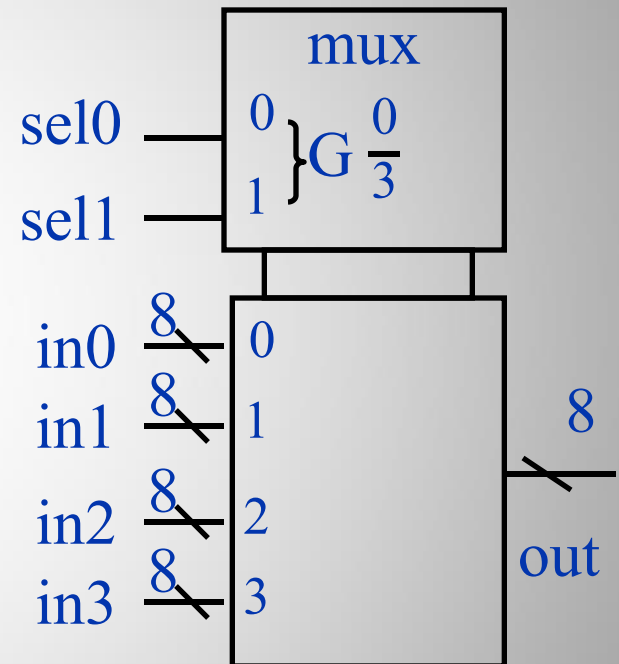
```
        else if(sel == 2'b01) out = in1;
```

```
        else if(sel == 2'b10) out = in2;
```

```
        else                  out = in3;
```

```
    end
```

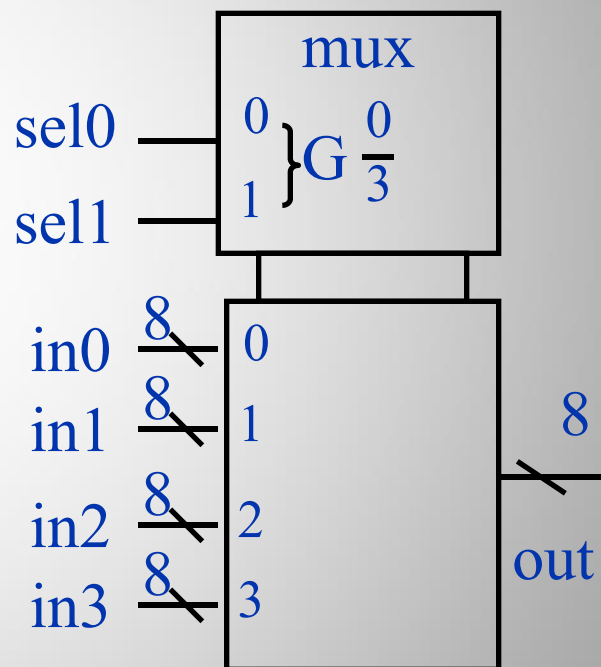
```
endmodule
```



用case语句描述4选1 MUX

```
module mux_4_1(out , in0 , in1 , in2 , in3 , sel);  
    output[7:0] out ;  
    input[7:0] in0 , in1 , in2 , in3 ;  
    input[1:0] sel;  
    reg[7:0] out ;  
    always @(in0 or in1 or in2 or in3 or sel)  
    begin  
        case(sel)  
            2'b00:  out = in0;  
            2'b01:  out = in1;  
            2'b10:  out = in2;  
            default: out = in3;  
        endcase  
    end  
endmodule
```

共有8个4选1 MUX,
它们共用sel信号



例6 奇偶校验器

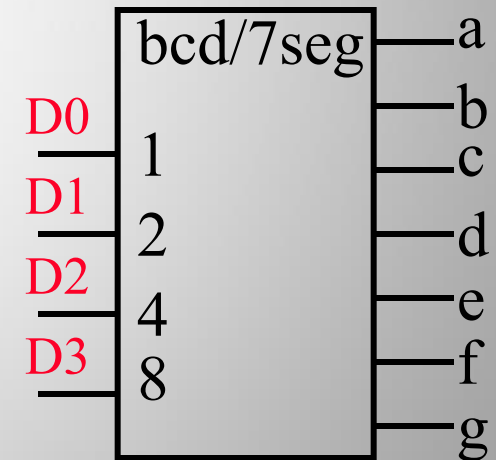
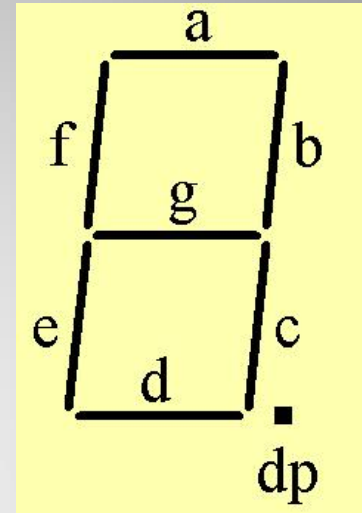
奇校验：输入数据“1”的个数为奇数。可用“异或”运算

偶校验：输入数据“1”的个数为偶数。可用“同或”运算

```
module parity(even_bit,odd_bit , input_bus);  
    output      even_bit,odd_bit ;  
    input [7:0]  input_bus;  
    assign odd_bit = ^ input_bus; //产生奇校验位  
    assign even_bit = ~odd_bit; //产生偶校验位  
endmodule
```

例7 BCD码-七段 译码器(共阴)

```
module decode4_7(a,b,c,d,e,f,g,D);  
    output a,b,c,d,e,f,g;  
    input[3:0] D;  
    reg a,b,c,d,e,f,g;  
    always @(D)  
        begin  
            case(D)  
                4'd0 : {a,b,c,d,e,f,g}=7'b1111110;  
                4'd1 : {a,b,c,d,e,f,g}=7'b0110000;  
                4'd2 : {a,b,c,d,e,f,g}=7'b1101101;  
                4'd3 : {a,b,c,d,e,f,g}=7'b1111001;  
                4'd4 : {a,b,c,d,e,f,g}=7'b0110011;  
                4'd5 : {a,b,c,d,e,f,g}=7'b1011011;  
                4'd6 : {a,b,c,d,e,f,g}=7'b1011111;  
                4'd7 : {a,b,c,d,e,f,g}=7'b1110000;  
                4'd8 : {a,b,c,d,e,f,g}=7'b1111111;  
                4'd9 : {a,b,c,d,e,f,g}=7'b1111011;  
                default : {a,b,c,d,e,f,g}=7'bx;  
            endcase  
        end  
    endmodule
```



例8 运算电路

例：带进位的 n 位通用加法器

```
module adder_n (cout, sum, ina, inb, cin);
```

```
    parameter n=16;
```

```
    output[n:1] sum;
```

```
    output cout;
```

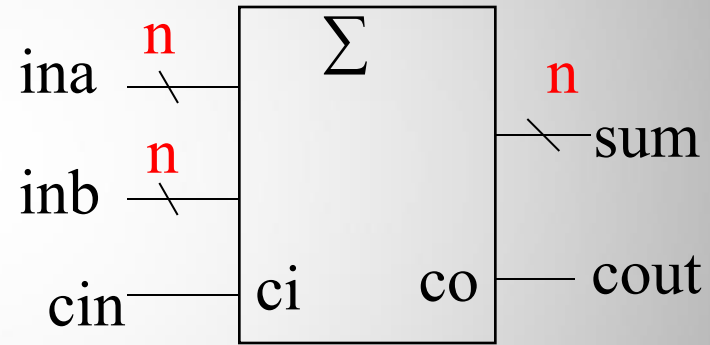
```
    input[n:1] ina,inb;
```

```
    input cin;
```

```
    assign {cout,sum}=ina+inb+cin;
```

```
endmodule
```

逻辑图



例9 比较器

```
module compare_n (great, equal, Small, ina, inb);
```

```
    parameter n=16;
```

```
    output great, equal, Small;
```

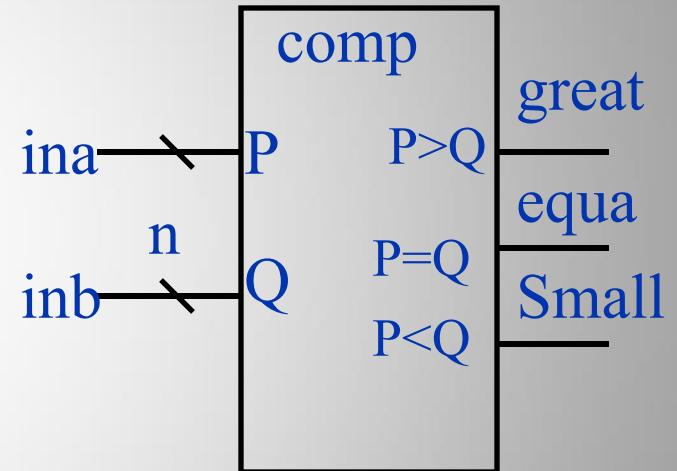
```
    input[n:1] ina,inb;
```

```
    assign great=(ina>inb);
```

```
    assign equal=(ina==inb);
```

```
    assign Small=(ina<inb);
```

```
endmodule
```



课后作业

- 练习：
 - 编译工具 modelsim、MAXplus等
- 作业：
 - 见/学在浙大；
- 阅读：
 - Verilog参考书的实例